Java Swing Final Project

From a real-life problem → OOP design → GUI → working application

TEAM

Duo only — 2 students

OUTPUT

Functional Java Swing application

FINISH

Documentation + demo + oral defense

Final Project Goal

Your project should solve a small, believable real-world problem.

- Design and develop a functional Java Swing desktop application.
- Apply object-oriented programming concepts in a meaningful way.
- Create a usable graphical user interface, not just a console program.
- Demonstrate event handling, validation, testing, debugging, and documentation.
- Work as a duo: both members must understand and be able to explain the system.

The Big Idea: Don’t Start With Code

Start with a problem, then design the solution.

1 OBSERVE

What everyday process is slow, confusing, repetitive, or easy to forget?

2 MODEL

What objects, data, actions, and rules exist in that process?

3 BUILD

Turn the model into Java classes + Swing screens + events.

Think like a software developer: What problem are we solving?

Avoid copying a tutorial project exactly. Your team should make its own decisions.

Project Challenge: Think of Your Own System

Use the examples below only as inspiration.

Washing Machine

Select cycle → set options → start → monitor status → finish

Self-Service Laundry

Customer → machine → load → payment → status → claim

Coffee Shop

Order → customize → calculate total → queue → release order

Parking System

Vehicle entry → slot assignment → time → fee → exit

Clinic Queue

Patient registration → queue number → service → status

School Borrowing

Item search → borrower → checkout → due date → return

Idea Generation Activity — Duo

10–15 minutes. No coding yet.

1. Each member lists 3 real-life processes they understand.
2. Compare your ideas and choose one shared problem.
3. Ask: Who uses it? What goes in? What happens? What comes out?
4. Define 3–5 core features only. Keep the scope realistic.
5. Give your system a name and write a one-sentence purpose.

RULE Do not choose a project just because it looks complicated. Choose one you can finish, test, and defend.

Turn the Idea Into a Problem Statement

WEAK

"We will make a washing machine app."

This describes the product, not the problem.

BETTER

"The application will simulate and manage washing cycles so users can select settings and monitor the machine state."

A good problem statement answers:

- Who is the user?
- What decision or action does the system support?
- What problem or process are you improving?
- What is the expected result?

Identify Your Objects (OOP Thinking)

Nouns in the problem often become candidate classes.

EXAMPLE: LAUNDRY

```
WashingMachine
Cycle
ClothingLoad
Customer
Payment
```

EXAMPLE: PARKING

```
Vehicle
ParkingSlot
Ticket
ParkingSession
Payment
```

ASK YOURSELF

What objects have:

- data?
- behavior?
- relationships?

Don’t create a class just to increase the number of classes. Each class should have a reason to exist.

Class Design: State

```
public class WashingMachine {
    private String mode;
    private int duration;
    private boolean running;

    public WashingMachine(String mode, int duration) {
        this.mode = mode;
        this.duration = duration;
        this.running = false;
    }
}
```

State = the fields an object remembers. The constructor sets the starting values.

Class Design: Behavior

```
public class WashingMachine {
    // fields and constructor: previous slide

    public void start() {
        running = true;
    }

    public void stop() {
        running = false;
    }
}
```

Behavior = the methods that change or report the object's state.

Encapsulation: Protect Your Data

- Keep important fields private.
- Expose controlled methods for reading or changing state.
- Validate values before changing an object’s state.
- Avoid putting all logic directly inside button event handlers.

```
private double balance;

public void addBalance(double amount) {
    if (amount > 0) {
        balance += amount;
    }
}

public double getBalance() {
    return balance;
}
```

Inheritance: Use It With Purpose

```
abstract class Payment {
    public abstract double calculateFee(double amount);
}

class CashPayment extends Payment {
    @Override
    public double calculateFee(double amount) {
        return 0;
    }
}
```

Use inheritance when there is a genuine “is-a” relationship and shared behavior—not just because the rubric mentions inheritance.

Polymorphism: Same Method, Different Result

```
class CashPayment extends Payment {
    @Override
    public double calculateFee(double amount) {
        return 0;
    }
}

class CardPayment extends Payment {
    @Override
    public double calculateFee(double amount) {
        return amount * 0.02;
    }
}
```

Each child class overrides calculateFee() in its own way.

Plan the Swing GUI Before Coding

Main Window

Navigation / dashboard / system status

Input Form

Text fields, combo boxes, radio buttons, checkboxes

Action Area

Buttons trigger operations

Output Area

Table, labels, status messages

Sketch the screen first. Decide what the user sees, enters, clicks, and receives.

Swing Components You Can Use

`JFrame`

main application window

`JPanel`

organize sections of the interface

`JLabel`

display text or status

`JTextField / JTextArea`

user input

`JButton`

trigger actions

`JComboBox`

choose from options

`JRadioButton / JCheckBox`

settings and selections

`JTable`

display records or transactions

`JOptionPane`

simple dialogs and confirmations

Event Handling: Connect the GUI to Your Logic

```
startButton.addActionListener(e -> {
    if (!machine.isReady()) {
        JOptionPane.showMessageDialog(
            this, "Please select a cycle.");
        return;
    }

    machine.start();
    statusLabel.setText("Machine is running...");
});
```

Good rule:



Separate the Responsibilities

MODEL

Java classes

- Stores state
- Applies business rules
- Performs calculations

VIEW

Swing components

- Displays information
- Collects input
- Shows feedback

CONTROLLER / LOGIC

Responds to events

- Coordinates actions
- Calls model methods

Goal: avoid one giant JFrame class containing every variable, calculation, and rule.

Validation: Make the App Difficult to Break

- Required fields must not be empty.
- Numbers must be numeric and within a sensible range.
- Invalid selections should produce clear feedback.
- Prevent impossible actions: e.g., starting a machine with no cycle selected.
- Confirm destructive or irreversible actions when appropriate.

TEST THIS

- Empty input
- Wrong input
- Extreme values
- Repeated clicks
- Cancel actions

Project Workflow: From Idea to Demo

1. Idea
2. Requirements
3. Class Design
4. GUI Prototype
5. Implement
6. Test
7. Document
8. Defend

Duo Work Distribution

Both members are responsible for the whole system.

MEMBER A — EXAMPLE ROLE

- Lead UI / Swing screens
- Implement selected classes
- Input validation
- GUI testing

But must understand the backend and logic.

MEMBER B — EXAMPLE ROLE

- Lead domain/OOP design
- Implement selected classes
- Business rules
- Integration testing

But must understand the GUI and events.

Roles are not excuses for “I don’t know that part.” During defense, either member may be asked about any feature.

Minimum Functional Requirements

- [ ] At least 3–5 meaningful features.
- [ ] At least 3–5 well-designed classes where appropriate.
- [ ] Use encapsulation and appropriate access modifiers.
- [ ] Use constructors and methods meaningfully.
- [ ] Use at least one meaningful relationship among classes.
- [ ] Functional Swing GUI with event handling.
- [ ] Input validation and user feedback.
- [ ] Tested application with documented test cases.

Documentation Checklist

- [ ] Project title and team members
- [ ] Problem statement and objectives
- [ ] System overview / scope
- [ ] Use-case or feature description
- [ ] Class diagram / UML
- [ ] GUI screenshots with brief explanations
- [ ] Important code snippets or design decisions
- [ ] Test cases and results
- [ ] Limitations and possible future improvements

Final Presentation / Defense

1. Introduce the problem and your solution.
2. Demonstrate the main workflow from start to finish.
3. Explain your important classes and OOP decisions.
4. Show one validation/error scenario.
5. Explain how the two members divided the work.
6. Answer questions about the code—not just the slides.

DEMO TIP

Do not only click buttons. Explain what the system is doing and why the code is designed that way.

Suggested Evaluation Criteria

Adjust percentages to match your official course rubric.

Weight	Criteria	What it covers
30%	**Functionality**	Application works and features are complete
25%	**OOP Application**	Classes, encapsulation, constructors, methods, relationships
15%	**GUI & Usability**	Clear layout, controls, feedback, user flow
10%	**Testing & Validation**	Handles normal and invalid scenarios
10%	**Documentation**	Complete, organized, technically accurate
10%	**Presentation & Defense**	Demo, explanation, individual understanding

Your Challenge Starts Now

Duo Project Planning Activity

A FIND A PROBLEM

Choose a real-life process. Do not start by searching for a code tutorial.

B DESIGN THE SOLUTION

Define users, features, objects, classes, and screens.

C PITCH IT

Explain your idea in 60 seconds. Problem → Solution → Features → OOP

REMEMBER **The best project is not the biggest one. It is the one you can explain, test, and finish.**

Quick Project Pitch Template

Project Name:

_____

Problem:

We noticed that

_____

_____

Main Users:

_____

Our Solution:

We will build a Java Swing application that

_____

_____

Core Features:

1. _____
2. _____
3. _____

Possible Classes:

_____

Before You Leave: Project Proposal

- ☐ Duo members identified
- ☐ Project title selected
- ☐ Real-life problem identified
- ☐ 3–5 core features listed
- ☐ Initial classes identified
- ☐ Rough GUI/screen flow planned
- ☐ Both members can explain the idea

NEXT SESSION

Bring your approved concept and start converting the idea into classes and Swing screens.